SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

B.Tech Computer Science and Engineering

BLOOD BANK MANAGEMENT SYSTEM

DCDS LAB ESE MINI-PROJECT REPORT

A Python-based application with SQL and NoSQL integration

Academic Detail	Information
Institute	Symbiosis Institute of Technology, Pune
Department	Computer Science and Engineering
Course	Database Concepts for Data Science Lab
Batch	2025–29
Year / Semester	SY / Semester III
Division	AIML-A
Member 1	Bivek Sah (PRN:- 25070126052)
Member 2	Alok Yadav (PRN:- 25070126022)
Member 3	Ayush Kumar Yadav (PRN:- 25070126048)
Submission	DCDS Lab ESE Mini-Project, October 2026

Abstract and Project Overview

The Blood Bank Management System is a database-driven application designed to organize donor details, donation records, blood-unit stock and hospital requests in one place. The proposed workflow aims to make blood-group availability easier to check, maintain traceable donation and request records, and highlight stock that needs attention. Structured entities such as donors, hospitals, donations, blood units and requests are stored in a relational SQL database. MongoDB is used for flexible supporting documents such as screening logs, blood-donation camp events and audit logs.

Python connects the database layer with application workflows. SQL supports joins, filtering, grouping and aggregate reporting; Python functions and classes manage validation and business rules; and MongoDB stores records whose fields may vary by event type. The design emphasizes data integrity, careful handling of personal information, and clear status transitions. This report describes the proposed system architecture and implementation plan. Screenshots, actual test outputs and final implementation details must be updated to match the working application before submission.

Project Objectives

- Maintain donor profiles and allow search by blood group and location, subject to consent and privacy rules.
- Record donations and connect eligible donations to blood-stock records.
- Track blood units by group, collection date, expiry date and status.
- Record hospital requests and track request fulfilment status.
- Summarize stock by blood group and flag low stock or units nearing expiry.
- Use SQL for structured transactional records and MongoDB for screening, camp and audit documents.
- Demonstrate Python functions, classes, validation, exception handling and file handling.

Keywords

Blood bank; donor management; blood inventory; SQL; SQLite/MySQL; MongoDB; Python; data validation; hospital requests; expiry monitoring.

1. Problem Statement and Motivation

Blood banks coordinate several connected activities: registering donors, recording donations, checking eligibility, tracking collected units and responding to requests from hospitals. If these activities are managed in disconnected spreadsheets or paper records, staff may spend additional time verifying details and identifying available units. Inconsistent updates can also cause inaccurate stock counts or make it difficult to trace the status of a request.

Blood is a time-sensitive resource. A stock total alone does not provide the complete picture: units have blood groups, collection dates, expiry dates and statuses. A useful system should distinguish available, reserved, issued, expired and discarded units. It should also show whether a request is pending, partially fulfilled, fulfilled or rejected.

1.1 Formal Problem Statement

To design a Python-based database application that maintains donor, donation, blood-stock, hospital and request records; uses SQL and NoSQL for complementary data needs; and provides validated workflows and summaries for blood availability, expiry exposure and request status.

1.2 Problems Addressed

Problem	Possible consequence	System response
Fragmented records	Staff must search multiple records to answer routine questions.	Centralized donor, donation, stock and request views.
Unclear availability	A total stock number may include unavailable or expired units.	Track unit status and expiry separately.
Expiry exposure	Units may approach expiry without being highlighted.	Calculate days remaining and assign risk bands.
Urgent hospital request	Blood-group availability may take time to verify.	Filter eligible available stock by blood group and status.
Weak traceability	It may be difficult to determine who changed a record.	Store event/audit documents in MongoDB.
Inconsistent data	Invalid quantities or missing details can affect reports.	Validate inputs and use transactions where appropriate.

1.3 Expected Outcome

The expected outcome is an integrated workflow in which structured records are validated, stored consistently, retrieved through SQL, supplemented with flexible event logs, and presented in a simple interface for review by authorized users.

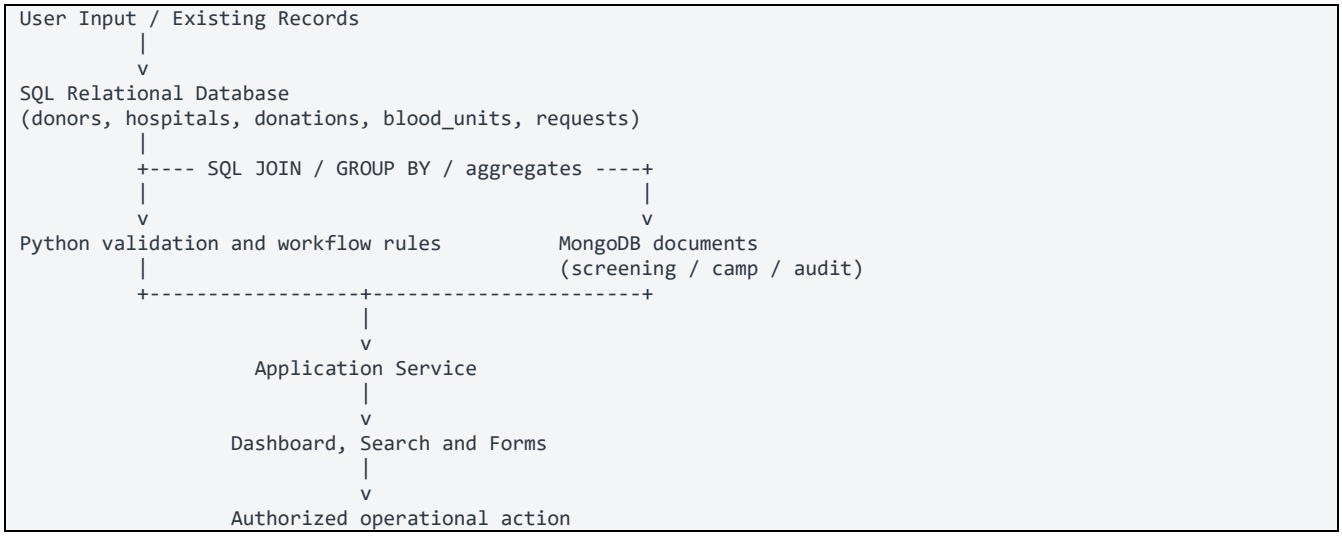
2. Overall System Pipeline

The pipeline starts with data capture. Donor and hospital master details are stored separately from donations and requests. Blood-stock records preserve the identity and state of each unit or batch, including blood group, collection date and expiry date. SQL queries then assemble the records needed for operational views and summary reports.

Python applies validation and business rules before or during database operations. MongoDB can store screening notes, camp-event information and audit events without forcing every event type into the same relational table. The user interface presents searchable lists, status summaries and expiry alerts. When an authorized user records a donation or issues units against a request, the underlying database is updated and the dashboard is refreshed.

Stage	Component	Responsibility
1	Data capture	Donor, hospital, donation, stock and request details.
2	SQL database	Store structured records and relationships.
3	SQL retrieval	Joins, filters, grouping and aggregate summaries.
4	Python service layer	Validation, transactions and workflow rules.
5	Analytics	Stock-by-group summary, low-stock indicators and expiry risk.
6	MongoDB	Screening logs, camp events and audit documents.
7	Application UI	Forms, search, stock tables and request status.
8	Feedback	Successful actions update stored records and refresh views.

2.1 Pipeline in One View



The design principle is to keep responsibilities clear: SQL is the source of truth for structured operational records, Python coordinates workflows, MongoDB supports flexible documents, and the interface makes information understandable.

3. Relational Database Design: SQL

A relational database is suitable for donors, donations, blood units, hospitals and requests because these entities have clear relationships and require consistent identifiers. Separating master data from transaction records reduces repeated information and supports reliable joins. The final schema should use primary keys, foreign keys, uniqueness constraints and suitable date/quantity checks.

3.1 Core Tables

Table	Key data	Purpose
donors	donor_id, name, contact, blood_group, city	Donor master record; restrict access to personal details.
hospitals	hospital_id, hospital_name, contact, address	Hospital / requesting organization details.
donations	donation_id, donor_id, donation_date, screening_status	Record of donation events linked to a donor.
blood_units	unit_id, donation_id, blood_group, collection_date, expiry_date, status	Unit-level stock and lifecycle status.
blood_requests	request_id, hospital_id, blood_group, quantity, request_date, status	Hospital demand and fulfilment status.
request_allocations	allocation_id, request_id, unit_id, issue_date	Link units issued to a particular request.

3.2 Relationships

- One donor may have multiple donation records over time.
- A donation may create one or more blood-unit records, depending on the implemented collection model.
- Each blood unit has a blood group, collection date, expiry date and current status.
- One hospital may submit multiple blood requests.
- Request allocations link a request to the units issued to fulfil it.

3.3 Design Decisions

Donor personal information should be collected only when required and visible only to authorized users. Donation eligibility and medical screening decisions must remain under qualified staff policy; the software should record approved outcomes rather than independently make medical decisions. Unit status should be changed through controlled workflows so that expired, discarded, reserved or issued units are not counted as freely available stock.

3.4 SQL Engine

SQLite is suitable for a small local academic prototype; MySQL or PostgreSQL may be preferred if the implementation requires concurrent multi-user access. The report and final code must name the database engine actually used. Database-specific features such as stored procedures must not be claimed unless the chosen engine and implementation support them.

4. SQL Operations and Data Retrieval

SQL is used to retrieve connected information from normalized tables. For example, a stock summary needs blood-unit data grouped by blood group and filtered by status and expiry. A request view may join hospital details with request status and allocated units. Queries should use parameterized values rather than concatenating user-provided strings.

4.1 JOIN: Units with Donation Information

```
SELECT
    bu.unit_id,
    bu.blood_group,
    bu.collection_date,
    bu.expiry_date,
    bu.status,
    d.donation_date
FROM blood_units AS bu
JOIN donations AS d
  ON bu.donation_id = d.donation_id
WHERE bu.status = ?;
```

4.2 GROUP BY: Available Stock by Blood Group

```
SELECT blood_group, COUNT(*) AS available_units
FROM blood_units
WHERE status = 'AVAILABLE'
  AND expiry_date >= CURRENT_DATE
GROUP BY blood_group
ORDER BY blood_group;
```

The exact date syntax may need adjustment for SQLite, MySQL or PostgreSQL. The query is illustrative and should be tested against the actual schema. Expired or non-available units should not be counted as available stock.

4.3 Request Summary

```
SELECT
    h.hospital_name,
    r.blood_group,
    r.quantity AS requested_quantity,
    r.status,
    r.request_date
FROM blood_requests AS r
JOIN hospitals AS h
  ON r.hospital_id = h.hospital_id
ORDER BY r.request_date DESC;
```

4.4 Querying Principles

- JOIN combines related donor, donation, unit, hospital and request information.
- COUNT and SUM produce stock and request summaries.
- GROUP BY creates blood-group and status-wise reports.
- WHERE filters by status, blood group, date and authorized workflow conditions.
- ORDER BY helps show recent requests or units with the nearest expiry date.
- LEFT JOIN can preserve requests or donors even when related records are absent.

5. Advanced Querying and Business Logic

Advanced querying is based on a business question that requires several steps: filtering eligible stock, grouping by blood group, checking expiry and comparing available units with a request. SQL performs relational retrieval and aggregation, while Python handles application-specific validation and multi-step workflows.

5.1 Advanced Query Patterns

Technique	Use in the system	Why it matters
Multi-table JOIN	Connect units to donations and requests to hospitals.	Produces complete operational views.
Aggregate functions	COUNT units; SUM requested/allocated quantity.	Measures stock and demand.
GROUP BY	Summarize stock by blood group and status.	Supports quick availability checks.
Date filtering	Find expired or soon-to-expire units.	Highlights time-sensitive stock.
Conditional logic	Classify stock and request status.	Turns stored values into clear states.
Sorting / ranking	Show nearest expiry or oldest pending request.	Helps staff prioritize review.
Nested / derived analysis	Compare available units with requested quantity.	Supports a transparent fulfilment decision.

5.2 Example: Expiry Risk

```
days_remaining = expiry_date - current_date

if days_remaining < 0:
    expiry_status = "EXPIRED"
elif days_remaining <= 7:
    expiry_status = "CRITICAL"
elif days_remaining <= 30:
    expiry_status = "HIGH"
elif days_remaining <= 60:
    expiry_status = "MEDIUM"
else:
    expiry_status = "LOW"
```

Expiry bands are configurable operational labels, not medical guidance. Actual blood-component shelf life depends on the component and approved storage rules. The implementation must use the correct expiry date supplied by authorized blood-bank processes.

5.3 Stored Procedure / Reusable Workflow

If MySQL or PostgreSQL is used, a stored procedure may be implemented for a controlled stock-issue workflow if it fits the course rubric. If SQLite is used, conventional server-side stored procedures are not available; the equivalent reusable operation should be implemented as a parameterized Python function and transaction. The final report must accurately describe the selected engine.

6. Python Architecture and Components

Python acts as the integration layer between the UI, SQL database, MongoDB and analytics. A modular structure keeps database access separate from validation and presentation. This makes it easier to test a database function without launching the UI and to change a screen without rewriting the schema.

Component	Responsibility
app.py	Application pages, navigation, forms and tables.
database/sql_db.py	Create and manage SQL database connections.
database/crud.py	Create, read and update donors, donations, units, hospitals and requests.
database/query_runner.py	Execute stock and request summary queries.
database/mongodb.py	Connect to MongoDB using environment configuration.
services/donation_service.py	Validate donation entry and coordinate stock creation.
services/request_service.py	Validate request and coordinate issue/allocation workflow.
analytics/stock_analysis.py	Summarize stock by blood group and expiry band.
utils/validation.py	Reusable input and date validation.

6.1 Python Concepts Demonstrated

- Functions organize repeated database and validation operations.
- Classes can represent donors, blood units, hospitals or requests when object-oriented design is used.
- Conditional statements implement status and expiry rules.
- Dictionaries and lists support form options and result processing.
- Exceptions provide readable feedback when a database or validation operation fails.
- Context-managed connections and transactions help release resources and keep related changes consistent.
- CSV or JSON file handling may support import/export if included in the final implementation.

6.2 Maintainability

Changing expiry thresholds or status labels should be localized to the relevant service or analytics function. The interface should call these functions instead of duplicating business rules in multiple screens.

7. Blood Stock Analytics

The stock dashboard should distinguish total recorded units from units that are actually available for issue. A unit is counted as available only when its status and expiry date satisfy the implemented policy. Stock summaries can be grouped by blood group and status to show both supply and operational restrictions.

7.1 Suggested Measures

Measure	Definition / calculation	Interpretation
Available units	Count units with AVAILABLE status and valid expiry.	Stock potentially available for allocation.
Expired units	Count units whose expiry date is before today and are not already disposed.	Requires review and proper handling.
Near-expiry units	Count eligible units with expiry inside a configured warning window.	Supports prioritization.
Pending requests	Count requests in PENDING or PARTIALLY_FULFILLED state.	Shows outstanding demand.
Group-wise balance	Available units by blood group.	Makes stock distribution visible.
Fulfilment gap	Requested quantity minus quantity allocated, with a lower bound of zero.	Shows the remaining quantity to address.

7.2 Suggested Stock Status Rules

Condition	Suggested label
Unit status is ISSUED	ISSUED / NOT AVAILABLE
Unit status is DISCARDED	DISCARDED / NOT AVAILABLE
Expiry date is before current date	EXPIRED / QUARANTINE REVIEW
Expiry within configured critical window	NEAR EXPIRY
Unit status is AVAILABLE and not near expiry	AVAILABLE

These labels are examples for software reporting. Blood-bank staff must follow institutional procedures for quarantine, expiry, release and disposal. The system must not override approved clinical or regulatory rules.

8. Expiry and Traceability

Expiry monitoring should be performed at the unit or batch level because units of the same blood group can have different collection and expiry dates. A single total by blood group can hide the fact that some units are near expiry while others have a longer remaining shelf life.

8.1 Expiry Classification

Days remaining	Example risk label	Suggested action
Below 0	EXPIRED	Exclude from available stock and follow authorized disposal procedure.
0–7	CRITICAL	Prompt review by authorized blood-bank staff.
8–30	HIGH	Prioritize review and allocation where policy permits.
31–60	MEDIUM	Monitor and plan according to approved policy.
Above 60	LOW	No immediate date-based alert under these example thresholds.

8.2 Traceability Fields

- Unit identifier and linked donation record.
- Collection date, expiry date and current unit status.
- Request and allocation references where a unit is issued.
- Timestamp and event type for relevant status changes.
- Authorized user or process identifier where authentication is implemented.

The warning thresholds shown here are illustrative. Actual storage and expiry rules must come from approved component-specific procedures and be verified by qualified staff.

9. Hospital Request and Allocation Workflow

A hospital request should be stored as a separate record with the requested blood group, quantity, request date, priority or urgency field if approved, and status. Allocation should be recorded as a traceable relationship between the request and the units issued. The application should prevent the same unit from being allocated twice.

9.1 Suggested Request States

State	Meaning
PENDING	Request received and not yet allocated.
PARTIALLY_FULFILLED	Some, but not all, requested units have been allocated.
FULFILLED	The requested quantity has been allocated according to the workflow.
REJECTED / CANCELLED	Request closed without fulfilment, with a reason recorded where applicable.

9.2 Controlled Allocation Steps

```
1. Select the hospital request.
2. Validate request status and requested quantity.
3. Retrieve eligible, available units for the requested blood group.
4. Exclude expired, reserved, issued or discarded units.
5. Confirm the units to be allocated under approved policy.
6. Record request allocations and update unit status in one transaction.
7. Recalculate fulfilled quantity and request status.
8. Commit; otherwise roll back and show a clear error.
```

Compatibility and clinical release decisions must follow approved blood-bank practice. This application design is for record management and must not substitute for laboratory testing or professional authorization.

10. NoSQL and MongoDB

MongoDB can complement the relational database by storing flexible documents whose fields differ by event type. SQL remains the source of truth for structured donor, donation, unit and request records. MongoDB documents may support screening event logs, camp event details and audit trails, subject to the institution's privacy and retention policies.

10.1 Suggested Collections

Collection	Purpose	Example fields
screening_logs	Record screening workflow events.	donor_id, event_time, outcome_code, staff_ref
camp_events	Store donation camp metadata.	camp_id, venue, date, organizer, notes
audit_logs	Trace selected record changes.	entity_type, entity_id, action, timestamp, actor_ref

10.2 Example Document

```
{
  "entity_type": "blood_unit",
  "entity_id": "UNIT-00042",
  "action": "STATUS_UPDATED",
  "old_status": "AVAILABLE",
  "new_status": "ISSUED",
  "timestamp": "2026-10-27T10:15:00",
  "actor_ref": "authorized_user_id"
}
```

The example is illustrative, not evidence of a running database. Avoid placing unnecessary donor personal or medical information in flexible logs. Connection strings and credentials should be read from environment variables and excluded from source control.

11. Bridging SQL, NoSQL and Python

The integration is demonstrated when a user action changes the relational data and the application subsequently presents updated information while maintaining appropriate event logs. For example, an authorized user records a donation; the SQL layer stores the donation and associated units; the service validates required fields; MongoDB may record a screening or audit event; and the dashboard refreshes stock summaries.

Step	Bridge	Result
1	UI → Python service	Input is received and validated.
2	Python → SQL	Donor, donation, unit or request records are inserted or updated.
3	SQL → query results	Related records and stock summaries are retrieved.
4	Python → analytics	Expiry bands, availability and fulfilment gaps are calculated.
5	Python → MongoDB	Flexible event/audit document is recorded where applicable.
6	Service → UI	Updated stock, requests and alerts are displayed.
7	UI action → database	Authorized changes affect the next refreshed view.

11.1 Separation of Responsibilities

SQL = structured operational records; MongoDB = flexible event and audit documents; Python = orchestration and validation; analytics = stock and expiry summaries; UI = user interaction and visibility.

12. Application Design

The proposed interface should separate operational tasks so users can move from an overview to a specific record without searching through one crowded screen. Navigation can be organized around Dashboard, Donors, Donations, Blood Stock, Hospital Requests and Reports. The final interface should use role-appropriate access and avoid exposing donor contact or screening details unnecessarily.

Page	Purpose	Main elements
Dashboard	Quick operational overview.	Available units by blood group, near-expiry counts, pending requests.
Donors	Maintain donor profiles.	Search, add/edit form, consent and validation fields.
Donations	Record donation events.	Donor lookup, donation date, screening status and outcome.
Blood Stock	Inspect units and expiry.	Unit identifier, blood group, dates, status and filters.
Hospital Requests	Track request progress.	Hospital, blood group, quantity, status and allocations.
Reports	Summaries for authorized users.	Group-wise stock, expiry categories and request status.

12.1 Screenshot Evidence

Insert genuine screenshots from the final running application in this section. Do not use mockups as proof of implementation.

Screenshot slot	What it should show
Figure 1 — Dashboard	Real stock summary and current request/expiry counts.
Figure 2 — Donor / Donation	Form or list with validated records; mask sensitive data.
Figure 3 — Blood Stock	Unit list with blood group, expiry date and status.
Figure 4 — Hospital Requests	Request status and allocation/fulfilment view.

13. CRUD, Transactions and Validation

Dynamic operations show that the application is connected to a database rather than displaying fixed examples. A donation or allocation may involve multiple related database changes, so the operation should be transactional where supported. If any step fails, the system should roll back the incomplete change and report the issue clearly.

13.1 Donation Workflow

```
Input donor and donation details
  -> validate required fields and date
  -> verify approved screening outcome is recorded
  -> insert donation record
  -> create/link blood-unit records as applicable
  -> record audit event if configured
  -> commit transaction
  -> refresh stock and donation history
```

13.2 Hospital Request Workflow

```
Select hospital and request blood group/quantity
  -> validate positive quantity and request details
  -> save request with PENDING status
  -> retrieve eligible stock for review
  -> authorized allocation updates units and request
  -> commit or roll back
  -> refresh request history and stock summary
```

13.3 Validation and Error Handling

Validation	Expected behavior
Missing required donor/hospital field	Reject the form and explain which field is missing.
Invalid quantity or date	Reject the operation before database update.
Duplicate identifier where uniqueness is required	Show a clear duplicate-record error.
Insufficient eligible stock	Do not allocate units; show available eligible quantity.
Unit expired or not AVAILABLE	Prevent normal allocation.
Database connection/query error	Show a readable error and avoid partial updates.
Successful operation	Show confirmation and refresh the relevant view.

14. Testing, Security and Rubric Coverage

Testing should be performed incrementally: first verify schema creation and sample data, then test database functions, then test the Python service layer, and finally exercise each UI workflow. The following list is a test plan; mark each item as passed only after running it against the final implementation.

14.1 Test Plan

Test case	Expected result	Status
Add donor with valid fields	Record is saved and searchable.	[Run / record result]
Reject incomplete donor form	Validation message; no partial record.	[Run / record result]
Record donation	Donation and associated unit records are linked correctly.	[Run / record result]
Stock summary by blood group	Counts include only eligible available units.	[Run / record result]
Near-expiry unit	Unit appears in the configured warning group.	[Run / record result]
Request allocation exceeds stock	Operation rejected; stock remains unchanged.	[Run / record result]
Successful allocation	Unit status and request status update consistently.	[Run / record result]
MongoDB unavailable	Graceful error or documented fallback behavior.	[Run / record result]

14.2 Security Practices

- Use parameterized SQL queries.
- Keep database credentials in environment variables or a protected configuration file.
- Do not commit .env files or real donor data to source control.
- Limit access to donor personal and screening information.
- Validate inputs in both the UI and backend service.
- Use transactions for related stock/request updates.
- Record audit events for important state changes where configured.
- Use synthetic test data in screenshots and demonstrations.

14.3 Rubric Mapping

Criterion	Marks	Evidence to include
Application design and functionality	2	Working pages, donor/donation/stock/request workflows.
Python programming	2	Modular functions/classes, validation, exceptions and file handling if implemented.
SQL and NoSQL integration	2	Relational records plus MongoDB event/audit documents connected through Python.
Advanced querying	2	JOIN, GROUP BY, aggregates, date filtering and derived stock/request analysis.
Code quality / documentation / output	1	Organized modules, comments, schema, screenshots and report.
Testing / validation / error handling	1	Completed test cases, constraints and exception feedback.

15. User Interface Evidence: Dashboard

This section should contain a screenshot from the final running application. The dashboard should make it clear that figures are retrieved from the database rather than typed manually. A suitable overview includes eligible available units by blood group, near-expiry units and pending requests.

[INSERT ACTUAL DASHBOARD SCREENSHOT HERE]

Figure 1. Blood Bank Management System dashboard — replace this placeholder with a genuine screenshot from the implemented application.

15.1 Interpretation

- Group-wise stock indicates the number of eligible available units.
- Near-expiry counts highlight units requiring authorized review.
- Pending-request counts show outstanding demand.
- Each KPI should be traceable to a query or defined calculation.

16. User Interface Evidence: Donors and Donations

The donor and donation screens should demonstrate data entry, record retrieval and validation. Use synthetic or anonymized records in the report. Do not expose real donor phone numbers, addresses or medical screening notes in screenshots.

[INSERT ACTUAL DONOR / DONATION SCREENSHOT HERE]

Figure 2. Donor and donation workflow — replace this placeholder with a genuine screenshot.

16.1 What This Screen Should Demonstrate

- Donor records can be created and retrieved.
- Donation records link to the correct donor identifier.
- Required fields and date formats are validated.
- Screening outcomes are recorded only through the approved workflow.
- Successful changes are reflected in the database and history view.

17. User Interface Evidence: Blood Stock and Hospital Requests

The blood-stock screen should show blood group, collection date, expiry date and current status. The request screen should show the hospital, requested group and quantity, request date and current fulfilment status. A screenshot should avoid showing sensitive data that is not required for evaluation.

[INSERT ACTUAL BLOOD STOCK SCREENSHOT HERE]

Figure 3. Blood stock and expiry view — replace this placeholder with a genuine screenshot.

[INSERT ACTUAL HOSPITAL REQUEST SCREENSHOT HERE]

Figure 4. Hospital request and allocation view — replace this placeholder with a genuine screenshot.

17.1 Interpretation

The two views should show how unit-level status affects available stock and how request fulfilment is traceable. A request should not be shown as fulfilled unless the implemented allocation records support that status.

18. Results, Limitations, Future Scope and Conclusion

The intended result is an integrated database application that organizes donors, donations, blood units, hospitals and requests. SQL supports structured relationships and summary queries; Python validates and coordinates workflows; and MongoDB supports flexible event or audit documents. The dashboard is expected to present stock-by-group, expiry risk and request status in a form that is easier to review than disconnected records.

18.1 Results

- Structured schema for donors, donations, stock, hospitals and requests.
- SQL joins and aggregate queries for stock and request summaries.
- Defined expiry-monitoring and availability rules.
- Proposed Python service workflows with validation and transactional updates.
- MongoDB collection design for screening, camp and audit events.
- Test plan and screenshot slots to document the final running implementation.

18.2 Limitations

- The current report is a design/template; actual behavior depends on the final implementation.
- Medical eligibility, compatibility and release decisions are outside the software's independent authority.
- Expiry rules must match the component type and approved blood-bank procedures.
- Authentication, authorization and deployment controls may require further development.
- Sample data and test outcomes must be replaced with verified results from the working application.

18.3 Future Scope

- Role-based access for administrators, blood-bank staff and hospital users.
- Automated low-stock and near-expiry notifications.
- Barcode/QR-based unit identification and traceability.
- Secure integration with authorized laboratory or hospital information systems.
- More detailed audit trails, backups and deployment monitoring.
- Reports on donation camps and demand trends using privacy-safe aggregated data.

18.4 Conclusion

The Blood Bank Management System provides a practical context for applying database concepts to a real-world coordination problem. Its proposed architecture combines relational integrity, SQL querying, Python business logic and flexible NoSQL records. Before submission, the report should be aligned with the actual source code, selected SQL engine, tested workflows, real screenshots and verified test outcomes.